From Feedback to Revision: When Do Multi-Agent Roles Look Load-Bearing?

Anonymous Author(s)
Submission Id: 2850

Abstract

Multi-agent large language model (LLM) systems usually assign roles such as Planner, Executor and Reviewer. We ask when these roles are load-bearing, that is, when removing them lowers performance while call and information budgets stay the same. We test three open-weight models on competition mathematics and two of them on college science. Keeping the agents, topology and budgets unchanged, we delete every role instruction. This deletion largely destroys the intended division of labor, yet first-pass accuracy does not drop. Roles appear useful for repairing rejected answers, clearly for the gemma-4 model and directionally for the gpt-oss model. But this advantage appears only when the role-free team never receives the evaluator's feedback. In an exploratory condition, we give the role-free team the same feedback string. It then recovers from failures more often than the role-structured team (recovery 0.609 against 0.222 for gemma-4 on competition mathematics) and reaches higher final accuracy in every setting. However, the advantage disappears when both teams are credited with the same number of evaluator verdicts, and one robustness check even favors roles. The difference in recovery mainly reflects how often each team submits. The role-structured Reviewer often withholds approval, so its team receives fewer verdicts. In a single exploratory 400-problem run for gemma-4, we submitted the team's answer in every cycle (one pass of the three agents) regardless of the Reviewer, leaving the role prompts unchanged. The role-structured team then recovered as well as the role-free team. We conclude that a clean division of labor is not evidence of adaptability. Role ablations must hold information channels fixed, not only budgets.

Keywords

Multi-agent systems, large language models, role specialization, division of labor, organizational structure, ablation

ACM Reference Format:
Anonymous Author(s). 2027. From Feedback to Revision: When Do Multi-Agent Roles Look Load-Bearing?. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), Hanoi, Vietnam, 3–7 May 2027*, IFAAMAS, 10 pages.

1 Introduction

Researchers usually present a multi-agent large language model system as a detailed diagram. It features components like a Planner that decomposes without solving, an Executor that carries out the plan, a Reviewer that critiques rather than re-derives (PER), and arrows for who hands what to whom [21, 37, 43]. This diagram is read as a mechanism that explains where the system's competence comes from and justifies the cost of multiple agents. In practice, however, what runs is simply a set of prompts handed to a language model. The model may honor the intended division of labor, ignore it or satisfy its surface constraints while solving the entire problem in every box. A simple accuracy score cannot distinguish between these scenarios. A pipeline whose agents have quietly collapsed into identical solvers returns the same final score as an organization that works exactly as drawn.

This ambiguity raises a critical question about which phases of the problem-solving process, and under what conditions, genuinely rely on multi-agent organization. A team must first produce an answer and, if that answer is rejected, produce a different one. We ask when the assigned roles are truly load-bearing during these distinct phases. We define a component as load-bearing if removing or altering it reproducibly degrades task performance under matched call and information budgets (§3). Matching the information budget is subtle because feedback reaches each agent through its prompt, and a role ablation inherently rewrites those exact channels. To test this, we hold the topology, actor model (the language model that plays all three agents), retry budget and evaluator fixed, and we vary the organization in both directions: we delete every role instruction and, separately, we enforce the division of labor at the interface, through typed message formats. Deleting the roles largely destroys the intended division of labor (aided by the fact that the role-free prompt asks each agent for its current answer). Yet, consistent with prior role ablations [10, 35], this functional collapse does not degrade first-pass accuracy.

After a rejected first answer, roles do seem to matter. The role-structured team recovers more failed trajectories, runs on a problem whose first answer was rejected, than a role-free baseline team, clearly for the gemma-4 model and directionally for the gpt-oss model. However, this difference is manufactured by uneven information access. The role-structured templates carry the evaluator's verdict (PASS or FAIL) and repair hint (a correction that must not reveal the answer) to every agent. In contrast, the role-free baseline template has no place for feedback, so those agents never see the verdict or the hint. Roles appear load-bearing for repair only when their prompts are the only channel carrying the feedback. When we introduce an exploratory condition where the role-free team also receives the same feedback string, that team actually recovers more failed trajectories than the role-structured team and achieves higher final accuracy across competition mathematics and, for two of the models, college science.

However, when both teams are credited with the same number of evaluator verdicts per problem, the role-free advantage disappears, and one robustness check favors roles. Roles mainly change how



This work is licensed under a Creative Commons Attribution International 4.0 License.

Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), M. Baldoni, F. Fang, W. Yeoh, N. Yorke-Smith (eds.), 3–7 May 2027, Hanoi, Vietnam. © 2027 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org).

often the team asks for external feedback, rather than how well it adapts. Specifically, the role-structured Reviewer withholds approval even before any feedback exists and, on competition mathematics, mostly where no clause forbids approval, so the team submits fewer attempts. Against a ground-truth evaluator and a capped budget, a withheld submission is a lost chance to recover. Overriding this Reviewer in a single exploratory 400-problem run for gemma-4, with the role prompts unchanged, raises the role-structured team's recovery to match the role-free team's on the same problems. Enforcing the division of labor at the interface also loosens the gate and raises recovery (resolvably only for the Llama-3.3 model), but 8% to 17% of enforced trajectories never finish, so end-to-end accuracy does not improve. These results show that a clean division of labor does not guarantee that a team acts on corrective information. Evaluating agent organization requires holding information channels fixed alongside budgets.

We summarize our contributions as follows:

(1) An operational definition of load-bearing multi-agent components, applied per phase through a budget-matched design that varies organization in both directions.
(2) Evidence that a role ablation which drops the feedback channel can manufacture an apparent role effect on repair (clearly for gemma-4, directionally for gpt-oss), and that the role-free advantage disappears once the feedback string and the number of verdicts are matched.
(3) An account of what roles do instead: they mainly act as a gate on submission, measured from the logs and tested by removing the gate in one exploratory run.
(4) A measurement point extending a recent separation of reviewer precision from critique uptake [44]: properties resembling good collaboration do not guarantee that corrective information changes the system's next action (§6).

2 Related Work

Roles in organizational models. Organization-centered multi-agent systems give roles a formal existence independent of their occupants. Moise$^+$ separates structural, functional and *deontic* specifications [24]. OperA makes role adoption an explicit social contract [7], and electronic institutions specify norms outside the agents [13], to be regimented (violation made impossible) or sanctioned [19]. An LLM role prompt is the degenerate case: functional and deontic clauses in prose, with nothing to make them bind. Our role-free condition keeps the positions, the communication graph and the turn order, but deletes these clauses. Conversely, our enforced condition regiments the functional part at the interface. In this paper, adaptability means revising an answer across attempts on the same problem, not reorganizing the team [22]. Teamwork models make repairing a failed joint plan a team obligation [41], which a prompt can state but not secure.

Role specialization and role drift. Prompted roles are known to be unreliable. Agents disobey the role specification across annotated traces [4], and *role drift* has a training-time account [3]. Role-label conditioning fails to induce the intended specialization [35], and agents collapse into highly similar rationales [36]. Differentiation can even arise *without* assigned roles [12]. Recent enforcement-side work makes role scope part of the system structure rather than advice in the prompt [15, 40]. While role drift and the decoupling of role quality from task performance are known [3, 44], we measure them under a controlled ablation.

Redundancy and parity. Dylan et al. [10] hold total LLM calls fixed and find an evolved Planner, Executor and Critic team no better than an evolved single agent. Zhang et al. [47] find that many reported multi-agent efficiency gains arise where one agent suffices. This objection is why we report a single-agent reference (§4). On the same premise, Chen et al. [6] prune redundant agents.

Feedback, revision and answer lock-in. Iterative refinement with self-generated or tool feedback [17, 30, 38] and multi-agent debate [9, 29] are standard patterns. Models often repeat earlier outputs under exact verifier feedback [25]. Under self-reflection, this failure is called Degeneration-of-Thought [29], and belief entrenchment is studied directly [33]. Huang et al. [23] report that self-correction degrades accuracy only in the *intrinsic* setting, where the model judges its own answer, and improves it when the correct label serves as a stopping rule [see also 27, 39]. Our loop is gated by such an oracle (an evaluator that knows the correct answer), where more attempts buy more coverage, the share of problems solved by at least one attempt [1]. This is the regime in which the role-structured team's restraint costs it. Closest to our lever, Chen et al. [5] vary only the *role label* on a byte-identical erroneous claim and move explicit-correction rates by 23 to 93 percentage points. However, they do so in a single agent without budget matching.

Coordination versus capability. Canedo [2] frames structured specification as a *capability equalizer* whose value falls with model strength. Theoretical [46] and complementary [8] accounts agree. Our enforced condition reproduces that direction only under a scoring rule that drops the trajectories that never finish. Under our pre-registered rule (fixed in writing before the data existed), it reverses for the gemma-4 model and vanishes for the other actors (§5.5).

Prior studies of one trace corpus, and critique uptake. Three studies report on one multi-agent trace corpus on the Omni-MATH benchmark [26, 44, 45], from which our single-agent comparator is also drawn. The closest study separates reviewer *precision* from critique *uptake* [44]. It finds that a pipeline reviewer flags real errors more precisely than broadcast-style peer discussion does. Yet its correct critique far less often changes the next candidate, and uptake improves when the critique is placed in the solver's working context. We ask the organizational counterpart: with the corrective signal held fixed, does the role structure itself make a team more adaptive? None of these studies manipulates the role structure, checks that the specified division of labor is realized, or matches feedback access across conditions. Elsewhere, the failure to revise is almost always measured in a single agent, leaving no organization to ablate.

3 Definitions and Measurement

3.1 What "load-bearing" means here

An organization is specified by a tuple: agents, an interaction order, a message-passing graph, a context-visibility policy, and a *role contract* per agent. A contract asserts that an agent performs some

function (MUST decompose the problem) or that it does not (MUST NOT state a final answer). Clauses of the second kind make the specification a *division* of labor. Because a clause is prose handed to a language model, the *realized* organization is not guaranteed to match it. Nothing records which clauses were honored.

Definition. A component of an organization is **load-bearing** if removing or altering it reproducibly degrades task performance *under matched call and information budgets*.

The budget clause ensures we test organization rather than scale. Removing roles alongside agents, calls or context confounds the evaluation. We apply this definition *per phase*, because a team must first produce an answer and then repair it if it is wrong. The information clause covers every channel through which a signal reaches an agent. In a prompted system, feedback arrives through the same templates that carry the roles. Consequently, a role ablation can change both the roles and the feedback channel. A contrast is matched only when it changes one of them (§4).

3.2 Measuring the realized organization

Functional collapse (primary). The first deliberation cycle (one pass of Planner, Executor and Reviewer) *collapses* if *all three* agents independently assert their own final answers. The pipeline then runs as three solvers in sequence rather than a division of labor. This metric targets the property the diagram asserts rather than what a hash can see. We measure it on the first cycle so it is not confounded with trajectory length. It is not applicable to the structurally enforced conditions because their schema forbids an answer field.

Exact output duplication (secondary, a lower bound). This occurs when two or more role messages are byte-identical, based on a `sha256` hash (a cryptographic digest) of the raw completion. It bounds collapse from below (supplement) and is never used to compare actors.

Answer authorship, and computation. We measure whether a Planner authors a final answer. Structural enforcement leaves the Planner no field in which to write one, so this measure serves as the manipulation check. Separately, we track whether a Planner abstains from *computing*, scored as containing no evaluated numeric equality (**P1a**, pre-registered). Because P1a scores the absence of a pattern, it rewards short messages. Therefore, it cannot reliably measure a division of labor, and we use it only as a fidelity check (§4.1).

First pass and recovery. First-pass accuracy grades the first answer submitted to the evaluator. It need not come from the first cycle: for the role-structured team, it may follow internal revision without external feedback. If a trajectory never submits an answer before the budget runs out, it is graded once on its final answer, and that grade is its first pass. *Recovery* is the fraction of *failed* trajectories, those whose first graded answer was wrong, that nonetheless finish correct. Because recovery conditions on a post-treatment variable (whether the first answer failed, which the intervention can change), recovery contrasts are descriptive [32]. We also report the feedback-matched contrast on unconditional final accuracy (§5.2). The verdict-matched analysis (§5.3) is conditional by construction.

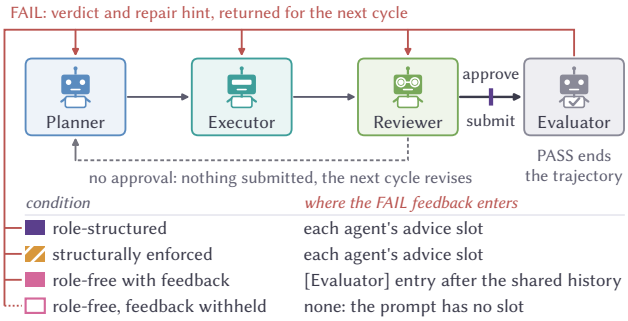


Figure 1: One deliberation cycle and where the feedback enters. A trajectory (one problem's run, at most six cycles) advances one cycle per pass of Planner, Executor and Reviewer. The third agent's approval (the bar marks this gate) is the only route to the evaluator, which knows the reference answer and returns PASS, ending the trajectory, or FAIL with a repair hint; without approval nothing is submitted and the next cycle revises. The key gives where the feedback enters per condition: the advice slot of each role prompt for the role-structured team (Planner, Executor and Reviewer prompts) and the structurally enforced one (the same roles imposed by typed message formats); for the role-free team (one generic prompt for all three agents, with no slot), either nowhere or as an `[Evaluator]` entry in the shared history. A trajectory that never submits is graded once when the budget runs out.

Validation. P1a was hand-adjudicated against a definition frozen *before* the adjudication sheet was opened. It clears a pre-set agreement threshold of 0.80 (supplement).

4 Intervention

We vary organization in both directions from one baseline: we remove it, and we enforce it. All conditions share three agents, one interaction order (Planner → Executor → Reviewer), the message-passing graph, context visibility (except under enforcement, below) and the total call budget. They also use the same verifier and prompt, retry budget, decoding parameters (temperature 0, that is, greedy decoding, and identical `max_tokens`), stopping rule (a PASS ends the trajectory) and grading. Every trajectory in the frozen snapshot carries the identical budget record, so the design tests organization, not scale. Besides the roles, three things differ. After a FAIL, the evaluator returns a repair hint, written with the gold answer (the reference solution) in view and instructed not to reveal it. The role-structured and enforced templates show the hint to all three agents in the next cycle, each through its role's advice slot, the template field reserved for the evaluator's feedback (Figure 1). The role-free template has no such slot, so we report the role-free condition both without and with the feedback. The role-free footer (the closing instruction of the prompt) also asks every agent for an answer, and the role-structured Reviewer carries two clauses the role-free prompt lacks.

Role-structured PER (baseline). Role prompts are as ordinarily written. The Reviewer is told to approve only a complete answer of the right shape. In a cycle that receives a FAIL, it is told to route the work back to the Planner or Executor rather than approve.

Role-free, feedback withheld. All three agents receive one byte-identical prompt, "You are one of three agents collaboratively solving this problem. Continue the work.", followed by the problem, the shared history and the agent's slot number (its position, first to third). A footer asks for the agent's current best answer in `\boxed{}` and a closing `[Agree]` (ready to submit) or `[Continue]`, the conventions the role-structured Executor and Reviewer use. Every clause saying who does what is removed, above all the Planner's MUST NOTs on computing and answering. In every prose condition, the third agent's `[Agree]` is approval (a message with neither tag counts as `[Continue]`), and the latest boxed answer is submitted. Nothing in this template carries the evaluator's verdict or hint.

Role-free with feedback. This condition uses the same role-free prompt, byte for byte, plus the evaluator's verdict and repair hint, the string the role-structured team receives. All three agents see it as an `[Evaluator]` entry after the shared history in the cycle after a FAIL, so it stays visible for one cycle, as the advice slot does. It differs from the feedback-withheld condition only in the feedback.

Structurally enforced. The interface, not the wording, enforces specialization. The Planner emits JSON (a typed output format) against a schema with *no answer field*, so a final answer is unrepresentable. The Executor receives the parsed plan object and the problem, never the Planner's raw text, and emits a derivation and an answer. The Reviewer emits a verdict, a target span and a critique. Its `accept` promotes the Executor's answer verbatim *by reference*, because it has no field in which to *author* one. Each function stays with one agent under the same budget, but the Executor sees less context, and a rejected output can exhaust the regeneration budget (the number of times a malformed output may be regenerated). A *context-restored control* repeats this condition with the shared history restored.

Single-agent reference. A budget-matched single-agent protocol on the same problems guards against multi-agent gains that merely mimic a single capable agent [47]. It comes from a separate, earlier corpus (§4.2). The comparison cuts both ways, and the arms come from separate runs (supplement).

4.1 Instrumentation and pre-registered analyses

An intervention that never engages cannot explain an outcome, so we measure engagement. We log every rejected-and-regenerated output with its reason, count only role-boundary violations in the enforcement rate, and count answer-bearing calls per condition. One logging gap slipped through (§5.1).

All five endpoints (the pre-specified analyses) were fixed in writing and committed before any intervention data existed. The full table, with the outcome for each, is in the supplement. Units are (problem, actor) pairs, and all conditions run the same problems, so every test is paired, with bootstrap intervals that resample whole problems [11]. The pre-registered primary endpoint, P4, asks whether enforcement's effect on accuracy differs by actor. It is null: no pairwise difference-in-differences (enforcement's effect in one actor minus its effect in another) survives Holm's correction for multiple comparisons [20] (smallest adjusted $p = 0.17$). Before correction, the interval for the Llama-3.3 model against gemma-4, +0.040 [+0.001, +0.080], only just excludes zero, and a separate resample gives $p = 0.055$. Only the gpt-oss against gemma-4 contrast had adequate statistical power for an interaction of plausible size.

The P1a manipulation objective also failed. It required enforcement to drive Planner computation-abstention to ≈1.0, but the schema forbids an *answer*, not a computation. It was re-designated to answer authorship. The role-structured versus role-free contrast this paper is organized around was pre-registered as a secondary, exploratory replication of a published null, and splitting it into phases is post hoc. The role-free condition with feedback was added after the freeze (when the dataset and analysis plan were fixed). Its design and primary analysis were committed in writing before any of its data existed, but its results are exploratory. Exclusions were fixed in advance, including the rule that decides §5.5: *unparseable trajectories count as incorrect rather than dropped.*

4.2 Campaign

Task and problems. We use the Omni-MATH benchmark of competition mathematics [14], sampled by a deterministic seeded manifest (a fixed, reproducible list of problem identifiers) so that every prefix of the run is a valid random sample. All conditions run the same problems, so every comparison is paired. A multiple-choice benchmark was ruled out before launch: on a four-option item, producing a different candidate succeeds one time in three by chance, which would inflate recovery.

Actors and scale. Three actor models are served from one pinned vLLM server [28]: gpt-oss-120b [34], gemma-4-31B [16] and Llama-3.3-70B [18]. The primary analysis reads one frozen snapshot of 11,600 trajectories, 83,320 role messages and 17,486 gradings, listed in a sha256 manifest, on 1,600 problems sampled uniformly at random from the 4,181 available. The three main conditions run 1,600 problems per cell (one actor in one condition on one domain) for gemma-4 and gpt-oss and 400 for Llama-3.3. The context-restored control covers gemma-4 and gpt-oss at 400 each. The evaluator is gpt-oss-120b in every cell, itself one of the actors under study (§7). The single-agent comparator is *not* part of this campaign; it comes from an earlier, separately collected corpus on the same benchmark, joined by problem identifier. Power simulated from that corpus is high for the paired per-actor main effect (§4.1 for the interaction). An early campaign, stopped when outputs cut off at the token limit were counted as role-boundary rejections, was never pooled (supplement).

The feedback-matched condition. It was run after the freeze (4,748 trajectories) on the same server, problems, budget and evaluator, for all three actors on Omni-MATH and for gemma-4 and gpt-oss on the SciBench benchmark [42]. Its data are manifested separately and never pooled into a frozen cell. The two gemma-4 cells were run a second time, independently. A re-run of gemma-4's role-structured condition on 400 frozen problems shows no detectable drift in final accuracy (0.818 against 0.825 frozen, paired $p = 0.75$; recovery 0.198 against 0.146). Decoding at temperature 0 is not deterministic

Table 1: Functional collapse on the Omni-MATH benchmark. Share of first cycles (one pass of Planner, Executor and Reviewer) in which all three agents assert their own final answer, per actor (the model playing all three agents). RS: role-structured (Planner, Executor and Reviewer prompts); RF⁻ and RF: role-free (one generic prompt for all three agents) with the evaluator's feedback withheld or shown. *Adjudicated no division*: the share of a judged sample of first cycles in which a model judge with a fixed rubric finds no division of labor; the judge did not see the grade but could infer the condition. The byte-duplication lower bound, the enforced condition and a sensitivity analysis over answer-marker definitions are in the supplement.

	functional collapse			adjudicated no division	
actor	RS	RF⁻	RF	RS	RF⁻
gemma-4-31B	0.000	**0.976**	**0.974**	0.000	**0.857**
gpt-oss-120b	0.120	**0.846**	**0.883**	0.000	**1.000**
Llama-3.3-70B	0.000	**0.872**	**0.900**	0.000	**1.000**

on a shared server. The two role-free conditions are byte-identical until the first verdict, yet they agree on gemma-4's first verdict for 1,496 of 1,598 problems, and their first-cycle collapse differs by up to 0.04 (Table 1). We therefore treat a cell as one sample and give the second run, where we have one, in the supplement. Prompts, runner and analysis scripts are in the supplementary material.

5 Results

All statistical contrasts are paired at the problem level, with bootstrap intervals clustered by problem. Trajectories that never produce a graded answer count as incorrect, except in the first-pass contrast of §5.1. Significance is the exact McNemar test [31] on discordant pairs (problems that exactly one condition gets right), with p-values to two significant figures.

5.1 R1. Removing roles changes the organization, not the first answer

We delete the role clauses, and the role-free prompt also asks every agent for its current best answer. The agents, interaction order, budget and evaluator are unchanged. We measure whether *every* agent asserts its own final answer in the first cycle, where trajectory length cannot confound the metric (Table 1). This rate rises from between 0.000 and 0.120 to between 0.846 and 0.976. Decomposed by role, the Planner supplies most of the change. The Executor answers in both conditions, and the Reviewer's rate, already high for two actors, rises in all three, but the Planner's moves from 0.000, 0.183 and 0.000 to 0.979, 0.898 and 0.889 (the gemma-4, gpt-oss and Llama-3.3 models). Since the footer asks for an answer, this measures what a Planner does once its MUST NOT clause is removed, not an emergent tendency; the claim concerns this *change*, not the absolute level. Under three answer-marker definitions the role-free rate stays between 0.846 and 0.999 while the role-structured rate reaches 0.892 only under the loosest marker, and the ordering holds in all nine actor by marker cells (supplement).

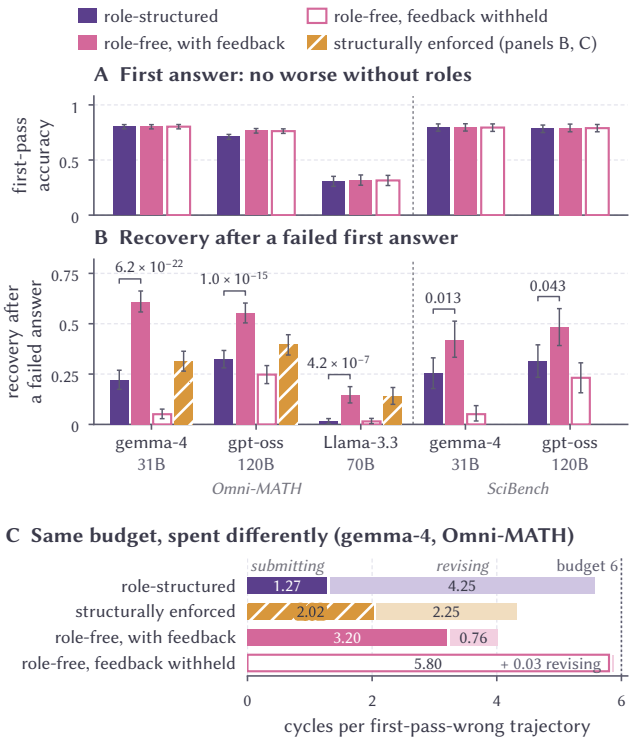


Figure 2: Roles organize the team; given the same feedback, they do not add recovery. Conditions: role-structured (Planner, Executor and Reviewer prompts); role-free (one generic prompt for all three agents) with the evaluator's feedback shown or withheld (hollow); structurally enforced (roles imposed by typed message formats; Omni-MATH only). Actors are the models playing all three agents. (A) First-pass accuracy: whether a team's first submitted answer is correct. (B) Recovery: the share of first-pass-wrong trajectories (runs whose first graded answer was wrong) that finish correct, each condition over its own set; brackets give the exact McNemar p for role-structured against role-free with feedback, paired on problems both first answer wrongly. (C) How gemma-4's first-pass-wrong Omni-MATH trajectories spend the six-cycle budget: cycles ending in a submission (the third agent approved) or a revision (it declined). (A) is over trajectories with a first verdict; in (B) trajectories that never finish count as unrecovered; error bars are bootstrap 95% intervals.

Content duplication and a second opinion. For gemma-4 and Llama-3.3, the role-free Planners write full independent solutions before boxing an answer, and the overlap between the numbers in their messages and those in the Executor's exceeds the overlap between Planner and Executor messages from different problems in the same condition by +0.449 and +0.432 (+0.159 and +0.145 with roles). A strong commercial model judging 171 trajectories against a frozen rubric agrees with the functional-collapse measure (Table 1; repeat agreement 0.953). This adjudication is outcome-blind (the judge did not see the grade) but not condition-blind: role words were renamed rather than equalized, and a renamed token

appears in all 42 role-structured items but only 3 of 56 role-free ones (Fisher's exact test, $p = 1.5 \times 10^{-24}$). The collapse claim does not rest on it.

First-pass accuracy does not follow. On problems graded under both conditions (n = 1,600 / 1,592 / 389), first-pass accuracy is *identical* for gemma-4 (+0.0000, [−0.0156, +0.0150]) and flat for Llama-3.3 (+0.0077), while it *improves* for gpt-oss (+0.0503, [+0.0339, +0.0666]; Figure 2A).

A logging defect. Our runner logged each response's `content` field, falling back to its `reasoning` field, the hidden reasoning trace, only when the content was empty. gpt-oss-120b often returns a short content string beside a long hidden trace, so its derivations are missing from the record: 75.7% of its role-free messages and 93.4% of its first cycles (the flag meant to report this reports 5.8%). Agents pass content only, by design and in every condition, so the defect is in the record alone, and no point estimate of gpt-oss's duplication is recoverable. It does not manufacture the collapse: in first cycles with no affected message, role-free collapse is 0.821 (n = 106) and role-structured 0.118.

5.2 R2. Given the same feedback, the role-free team recovers more

What a cycle is. The pipeline runs a fixed grid of six deliberation cycles, three outer rounds of two inner rounds, each one pass of Planner, Executor and Reviewer; Figure 1 shows how a cycle ends in a *submission* or a *revision*.

Recovery and final accuracy. The role-structured team receives the evaluator's feedback, so the like-for-like comparison is with the role-free team that also receives it (Table 2). That team recovers more: 0.609 against 0.222 (marginal, that is, each team over its own failed trajectories; Figure 2B). On the 236 problems both teams first answer wrongly, 89 are recovered only without roles and 4 only with them ($p = 6.2 \times 10^{-22}$). Both reach the same first-pass rate, and a correct first answer is never lost, so this carries into final accuracy: 0.921 against 0.846 ($p = 1.7 \times 10^{-24}$, conditioned on nothing). The recoveries are revisions, not re-gradings: in 192 of the 193, the passing answer differs from the one first rejected (541 of 547 across the five feedback-matched cells), and repeated identical evaluator prompts disagree in only 0.0072 of cases.

Reproducibility and leakage. A second, independent run of the two gemma-4 feedback-matched cells is close to the first, though the paired test on the SciBench benchmark weakens from $p = 0.013$ to 0.052; we report the first run throughout (supplement). Repair hints contain the gold answer verbatim in at most 2.3% of failing verdicts in any cell, and dropping every gemma-4 trajectory on the Omni-MATH benchmark that saw one leaves recovery at 0.599 and 0.212. We did not audit partial leakage.

Scope across actors. Both other actors repeat the ordering in recovery and final accuracy (Table 2), and in both the feedback leaves first-pass accuracy where it was (0.765 and 0.317, against 0.762 and 0.315 with it withheld). Llama-3.3's role-structured team rarely submits; there, enforcement recovers about as much as the role-free team with feedback (0.139 against 0.147 marginally; paired 9 against 17 of 219, $p = 0.17$). Against the feedback-withheld baseline, the role-structured advantage is clear only for gemma-4. It is directional for gpt-oss (0.324 against 0.247; paired 52 against 39 of 327, $p = 0.21$) and absent for Llama-3.3 (0.014 against 0.015). gpt-oss recovers without feedback, consistent with its answer moving from cycle to cycle anyway: one distinct answer in 20.8% of its failed trajectories, against 84.5% for gemma-4.

5.3 R3. Matched on verdicts, the gap closes; roles gate submission

The feedback-withheld baseline. Without the feedback, the role-free team turns nearly its whole budget into submissions. Across gemma-4's failed trajectories, it submits in 5.80 of 5.83 cycles (Figure 2C), its third agent never declines in 98.1% of these trajectories, and in 84.5% the transcript holds exactly one distinct answer. Nothing in its prompts carries the feedback, so each FAIL is followed by another cycle over the same history. Given the verdict and hint, the identical prompts produce a different team. The answer moves (one distinct answer in 6.6% of failed trajectories), and recovery rises from 0.051 to 0.609. Of the 265 problems both first answer wrongly, 139 are recovered only with the feedback and 2 only without it ($p = 7.2 \times 10^{-39}$). First-pass accuracy is 0.80 in both conditions, and 7 of the 16 recoveries without feedback are the same answer graded again. For gemma-4, revision requires the feedback rather than the roles.

Matched on verdicts, the gap closes. The role-free team with feedback also receives more of it: each submission returns a verdict and, on a FAIL, a hint written with the gold answer in view. To compare the teams at equal exposure, a post-hoc analysis takes the problems both first answer wrongly and cuts each role-free trajectory off after as many evaluator verdicts as the role-structured team received on the same problem. No cell then shows a significant difference (Table 2; $p \geq 0.48$), and Llama-3.3's role-structured team submits so rarely that its cell has five events. Pooled and stratified by cell, the difference is +0.009 in favor of roles (95% paired bootstrap confidence interval −0.014 to +0.033). The match is coarse: where the role-structured team received one verdict, the pair ties at zero by construction (141 of gemma-4's 236 pairs). Restricted to problems where it received at least two, the difference is +0.018 (−0.029 to +0.065); recoding the scoring defect of §7 gives +0.029 (+0.004 to +0.054; supplement). Because the comparison crosses the freeze, we repeat it against the post-freeze role-structured re-run. On the 65 problems both first answer wrongly, the role-free team leads 22 to 3 unmatched ($p = 1.6 \times 10^{-4}$), but roles lead 4 to 1 verdict-matched ($p = 0.38$). No variant puts the role-free team significantly ahead verdict for verdict; the only significant variant favors roles, by a small margin. The role-free team recovers more because it asks for more verdicts.

What roles do instead: a gate on submission. The gap in verdicts comes from the role-structured Reviewer. Across gemma-4's failed trajectories, that team submits 1.27 times in 5.52 cycles, against 3.20 in 3.96 without roles (Figure 2C). Of its declining cycles, 53% come before the first submission, when no feedback exists, and 84 of its 316 failed trajectories never submit within the budget; they are graded once at the end and never see the feedback. The clause forbidding approval in a cycle that receives a FAIL covers

Table 2: The feedback-matched contrast in every cell (one actor, the model playing all three agents, on one benchmark; n problems). RS: role-structured (Planner, Executor and Reviewer prompts); RF: role-free (one generic prompt for all three agents) with the evaluator's feedback shown; RF$^-$: role-free with it withheld. A trajectory (one problem's run) is first-pass wrong when its first submitted answer is graded wrong. *Recovery*: share of first-pass-wrong trajectories that finish correct, each condition over its own set. *Recovered only by*: on the n problems both RS and RF first answer wrongly, those recovered by one condition and not the other, with the exact McNemar p for the two counts. *Verdict-matched*: the same paired recovery with each RF trajectory cut off after as many evaluator verdicts (PASS or FAIL decisions) as RS received on that problem, so both get the same number of verdicts and hints. *Final accuracy*: correct at the end, over all problems, paired; never-graded trajectories count as incorrect. *Submissions*: answers sent to the evaluator per first-pass-wrong trajectory. *RS never submits*: RS first-pass-wrong trajectories with no submission in six cycles, graded once at the end.

	recovery			recovered only by (paired)				verdict-matched			final accuracy			submissions		RS never
actor, domain (n)	RS	RF	RF$^-$	n	RS	RF	p	RS	RF	p	RS	RF	p	RS	RF	submits
gemma-4, Omni-MATH (1,600)	0.222	0.609	0.051	236	4	89	6.2×10^{-22}	0.165	0.153	0.69	0.846	0.921	1.7×10^{-24}	1.27	3.20	84/316
gpt-oss, Omni-MATH (1,600)	0.324	0.552	0.247	319	17	101	1.0×10^{-15}	0.229	0.216	0.72	0.803	0.894	2.7×10^{-28}	2.10	3.53	51/463
Llama-3.3, Omni-MATH (400)	0.014	0.147	0.015	250	1	26	4.2×10^{-7}	0.012	0.008	1.0	0.315	0.415	1.6×10^{-7}	0.32	2.71	205/276
gemma-4, SciBench (574)	0.254	0.419	0.051	100	7	21	0.013	0.230	0.190	0.48	0.847	0.880	0.0054	1.95	3.62	3/118
gpt-oss, SciBench (574)	0.315	0.483	0.231	93	7	18	0.043	0.215	0.247	0.68	0.852	0.892	1.9×10^{-4}	2.68	3.95	1/124

only 19% of the declines. In later cycles that receive no FAIL, where approval is allowed, the Reviewer still declines in 75% of 504 such cycles, against 4% of 184 for the role-free team's third agent. On Omni-MATH, the clause covers 37% of gpt-oss's declines and 4% of Llama-3.3's. On SciBench, it covers 40% of gemma-4's declines and 60% of gpt-oss's (supplement). The gate might look like precision, since recoveries per verdict after the first are higher with roles (0.41 against 0.28 for gemma-4), but that compares different trajectories; on the same problem and the same number of verdicts, the two teams recover about equally.

Removing the gate alone. A direct, exploratory test keeps the role-structured prompts byte for byte but submits the team's answer in every cycle, whatever the Reviewer decides. We ran it once, for gemma-4 on the 400 Omni-MATH problems of the post-freeze re-run (supplement). First-pass accuracy is unchanged (0.768 against 0.773), the team draws 4.20 verdicts per failed trajectory against 1.44 for the gated re-run, and recovery rises from 0.198 to 0.613, as high as the role-free team's 0.549 on the same problems. On problems both first answer wrongly, the forced-submission team recovers 26 that the gated one misses and 2 the other way ($p = 3.0 \times 10^{-6}$); against the role-free team, the split is 7 to 7 ($p = 1$). With the role contract intact, removing the gate removed the gap.

5.4 R4. The same ordering in a second domain

SciBench's complete text-only slice of 574 free-response college science problems, frozen separately, tests whether the pattern belongs to the organization or to competition mathematics. The role-free prompt is byte-identical to its Omni-MATH form; the role-structured prompt keeps the same structural contract with domain wording adapted, as does the evaluator prompt. For both actors the ordering returns (Table 2): first-pass accuracy is flat, recovery is highest with feedback and no roles, and final accuracy is higher without roles. The recovery contrast is weaker and, as on Omni-MATH, vanishes when both teams are credited with the same number of verdicts. One behavior does not transfer: gemma-4's role-structured answer does not lock, that is, stay the same across cycles (one distinct answer in 7.6% of failed trajectories, against 8.5% for the role-free team with feedback), and by the second verdict the two recover alike (0.237 against 0.239; supplement).

5.5 R5. Structural enforcement shifts the allocation, and introduces brittleness

Enforced and role-structured prompts carry the same advice slot, so this contrast is matched on feedback access. Enforcement replaces the behavioral request with a constraint: the Planner's final answer becomes unrepresentable. The manipulation check passes. The Planner asserts a final answer in 0.000 of enforced first cycles in all three actors, against 0.979, 0.898 and 0.889 without roles, and enforcement logs 838, 1,183 and 395 role-boundary rejections per cell. It does not change what the Planner *computes*, so the adjudication still finds no functional division in 0.071 to 0.188 of enforced first cycles: an agent can be structurally stopped from answering without being stopped from doing the work. Relative to the role-structured team, enforcement loosens the gate. The enforced team submits more often (2.02, 2.33 and 1.84 per failed trajectory against 1.27, 2.10 and 0.32 for gemma-4, gpt-oss and Llama-3.3) and, for gemma-4 and Llama-3.3, unlocks the answer: one distinct answer in 0.119 and 0.072 of failed trajectories against 0.288 and 0.750 with roles (for gpt-oss, 0.092 against 0.052).

Recovery follows this loosening most clearly for the weakest actor: Llama-3.3's rises about tenfold, from 0.014 to 0.139 ($p = 3.8\times10^{-6}$). For gemma-4 (0.314 against 0.222, $p = 0.30$) and gpt-oss (0.395 against 0.324, $p = 0.11$), the direction is the same but not resolvable against attrition (trajectories that never finish). Recovery counts every trajectory with a recorded incorrect first verdict, including the 150 enforced trajectories that then died in regeneration, scored 0 by the pre-registration.

Improved revisability does not become end-to-end accuracy, because enforcement is brittle. Under the pre-registered rule, the accuracy difference is −0.0225, −0.0050 and +0.0175 for gemma-4, gpt-oss and Llama-3.3: one significant negative and two nulls. Of enforced trajectories, 8.0%, 9.9% and 17.0% never finish, against 0.0 to 0.8% role-structured and 0.0 to 2.5% role-free. Dropping them

gives +0.0190, +0.0361 and +0.0453: all positive, all significant, ordered by actor weakness, matching the direction reported for the capability equalizer [2]. That second reading is survivorship bias: it scores only the trajectories that finished, whereas the pre-registration fixed in advance that the others count as incorrect. Restoring shared history at fixed enforcement raises gemma-4's pre-registered accuracy (+0.0450, $p = 0.0079$; as graded +0.0196, $p = 0.14$) but not gpt-oss's (−0.0125, $p = 0.56$; both scorings with intervals in the supplement).

6 Discussion

Function of roles: gating submission vs. driving revision. Producing a first answer is a *generation* problem. Having three agents reach the same place by three routes costs nothing but tokens, and first-pass accuracy is nearly indifferent to the division of labor. After a failure, the team must produce something unlike the answer that just failed. Except for the gpt-oss model, teams rarely do so unless told that the candidate failed. Given that knowledge, the role contract adds little capacity to revise. When matched on the number of verdicts, the two teams differ little. The role contract primarily decides *when* the team asks for evaluation. The role-structured Reviewer withholds approval until it is satisfied, so the team draws fewer verdicts and sometimes none at all. Overriding the Reviewer closed the gap for the gemma-4 model. Under a ground-truth evaluator and a fixed cycle budget, fewer verdicts mean fewer recoveries. Where verification is costly, noisy, or unavailable, a team that asks less often could be the better design; this experiment cannot tell.

Practical implications for multi-agent system design. First, if a cheap and reliable verifier is available, a Reviewer role that withholds submission can cost recoveries. It is better to let the team submit and gate on the verifier rather than on an internal critic. Second, regardless of the role structure, the verifier's feedback must be routed to every agent that can act on it. In our pipeline, the teams of the gemma-4 and Llama-3.3 models without a feedback slot almost never recovered, whereas the same prompts recovered far more often once the feedback was added.

Methodological pitfalls: the necessity of information audits. In a prompted multi-agent system, feedback, tool outputs and verifier verdicts reach the agents through slots in the same templates that carry the role instructions, so rewriting those templates can silently remove a channel. Nothing in the budget record, call counts or accuracy metrics reports the absence. A role ablation therefore needs an information audit alongside the budget audit: the runtime inputs each template consumes must be documented, and a contrast is matched only when those input sets are identical. Our role-structured and feedback-withheld conditions differ in one such input, the feedback itself.

Reassessing the value of role diagrams. Precise reviewing does not necessarily mean the critique is adopted [44]. Similarly, a clean division of labor does not guarantee that a team adapts. What matters is whether corrective information changes the next action taken by the system. Task-outcome evaluation alone cannot test this because an aggregate score averages stages where the organization is idle with stages where it does all its work. We recommend two additions to standard evaluations: report performance conditioned on a failed first attempt, and measure the realized organization rather than assume the specified one.

7 Limitations

Evaluation paradigm and oracle-gated recovery. The recovery loop in our experiments is gated by an oracle. The evaluator triggering retries sees the ground truth, so every reported recovery rests on a ground-truth-informed failure signal rather than unaided self-correction. Stopping at the correct answer also hides revisions that would have turned a correct answer into a wrong one. An evaluator that can be queried multiple times favors the team that queries it most. As shown in §5.3, when matched on the number of verdicts, the role-free advantage disappears.

Scope, provenance, and experimental disclosures. We test one PER organization, with an evaluator that is itself one of the models under study. While the ordering returns across a second domain of college science problems, the findings are limited to text-based reasoning tasks. The feedback-matched condition and the forced-submission run were added after the freeze and are outside the pre-registration. The forced-submission run covers only one actor, one domain, and a single run, and it removes the entire gate rather than only the clause forbidding approval after a failure. The feedback-matched condition also differs in feedback presentation: role-structured agents receive feedback inside role-specific instructions, whereas role-free agents view it as a bare `[Evaluator]` entry. Finally, the context-restored control is only a *partial* control due to one-sided attrition (§5.5), and our functional measure only detects whether every agent asserted an answer, not whether their reasoning was identical.

A scoring defect. The automated runner graded the last *submitted* answer but recorded the last *held* answer. For the gemma-4 model on the Omni-MATH benchmark, the two disagree on 17% of role-structured first-pass-wrong trajectories, and on more in some other cells (per-cell rates in the supplement). Recoding the affected trajectories narrows the unadjusted recovery gap and shifts the pooled verdict-matched difference to +0.029 [+0.004, +0.054], which significantly favors the role-structured team (§5.3). The primary analysis keeps the scoring rule that was fixed before the data were collected.

8 Conclusion

Multi-agent roles organize a team into a structured workflow. Deleting these roles destroyed the intended division of labor across all tested models, but first-pass accuracy did not fall. Roles only appeared load-bearing for repairing rejected answers when compared against a role-free team that never received the evaluator's feedback. When both teams received the identical feedback string, the role-free team recovered more trajectories across every tested configuration. When matched on the number of evaluator verdicts, this advantage disappeared.

These findings suggest that roles change little about how a team uses corrective feedback but substantially alter how often the team asks for it. Structure is not adaptability. Evaluating agent organization therefore requires checking both whether the specified division of labor is realized and whether corrective information reaches the team and changes its next actions.

References

[1] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. 2024. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787 [cs.LG] doi:10.48550/arXiv.2407.21787

[2] Arquimedes Canedo. 2026. Architecture as Capability Equalizer for Coding Agents. arXiv:2608.21747 [cs.SE] doi:10.48550/arXiv.2608.21747

[3] Xiaoyang Cao, Siddarth Srinivasan, and Michiel A. Bakker. 2026. Do Modules Stay in Their Lane? Role Drift in Compound LLM Systems. arXiv:2607.21627 [cs.AI] doi:10.48550/arXiv.2607.21627

[4] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail?. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. arXiv:2503.13657

[5] Kuan-Yen Chen, Fang-Yi Su, Shih-Yen Lin, Bao Li, and Jung-Hsien Chiang. 2026. The Self-Correction Illusion: Role Relabeling Gates Explicit Error Flagging in Large Language Models. arXiv:2606.05976 [cs.AI] doi:10.48550/arXiv.2606.05976

[6] Yulang Chen, Haoxuan Peng, Jinyan Liu, Zichen Wen, Dongrui Liu, and Linfeng Zhang. 2026. AgentSlimming: Towards Efficient and Cost-Aware Multi-Agent Systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 30064–30086. arXiv:2605.08813 doi:10.18653/v1/2026.acl-long.1387

[7] Virginia Dignum. 2004. *A Model for Organizational Interaction: Based on Agents, Founded in Logic*. Ph. D. Dissertation. Utrecht University, Utrecht, The Netherlands.

[8] Victoria Dochkina. 2026. Drop the Hierarchy and Roles: How Self-Organizing LLM Agents Outperform Designed Structures. arXiv:2603.28990 [cs.AI] doi:10.48550/arXiv.2603.28990

[9] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 11733–11763. arXiv:2305.14325

[10] David Dylan, Aoife Brennan, Cian Murphy, Niamh O'Sullivan, Conor Kelly, and Saoirse Walsh. 2026. At Equal Inference Cost, Multi-Agent Structure Does Not Beat a Single Frozen Agent. arXiv:2609.04217 [cs.MA] doi:10.48550/arXiv.2609.04217

[11] Bradley Efron. 1979. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26. doi:10.1214/aos/1176344552

[12] Houssam El Kandoussi. 2026. "Who Am I, and Who Else Is Here?" Behavioral Differentiation Without Role Assignment in Multi-Agent LLM Systems. arXiv:2604.00026 [cs.CL] doi:10.48550/arXiv.2604.00026

[13] Marc Esteva, Juan-Antonio Rodríguez-Aguilar, Carles Sierra, Pere Garcia, and Josep L. Arcos. 2001. On the Formal Specification of Electronic Institutions. In *Agent Mediated Electronic Commerce: The European AgentLink Perspective (Lecture Notes in Computer Science, Vol. 1991)*. Springer, 126–147. doi:10.1007/3-540-44682-6_8

[14] Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, Zhengyang Tang, Benyou Wang, Daoguang Zan, Shanghaoran Quan, Ge Zhang, Lei Sha, Yichang Zhang, Xuancheng Ren, Tianyu Liu, and Baobao Chang. 2025. Omni-MATH: A Universal Olympiad Level Mathematic Benchmark for Large Language Models. In *International Conference on Learning Representations*. 100540–100569. arXiv:2410.07985

[15] Ling-Yue Ge and Lan-Zhe Guo. 2026. Roles with Rails: Contract-Preserving Role Evolution in Multi-Agent Structured Reasoning. arXiv:2605.28433 [cs.CL] doi:10.48550/arXiv.2605.28433

[16] Gemma Team. 2026. Gemma 4 Technical Report. arXiv:2607.02770 [cs.CL] doi:10.48550/arXiv.2607.02770

[17] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*. 57734–57811. arXiv:2305.11738

[18] Aaron Grattafiori et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] doi:10.48550/arXiv.2407.21783

[19] Davide Grossi, Huib Aldewereld, and Frank Dignum. 2007. Ubi Lex, Ibi Poena: Designing Norm Enforcement in E-Institutions. In *Coordination, Organizations, Institutions, and Norms in Agent Systems II (Lecture Notes in Computer Science, Vol. 4386)*. Springer, 101–114. doi:10.1007/978-3-540-74459-7_7

[20] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.

[21] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations*. 23247–23275. arXiv:2308.00352

[22] Bryan Horling and Victor Lesser. 2004. A survey of multi-agent organizational paradigms. *The Knowledge Engineering Review* 19, 4 (2004), 281–316. doi:10.1017/S0269888905000317

[23] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Learning Representations*. arXiv:2310.01798

[24] Jomi Fred Hübner, Jaime Simão Sichman, and Olivier Boissier. 2002. A Model for the Structural, Functional, and Deontic Specification of Organizations in Multiagent Systems. In *Advances in Artificial Intelligence: 16th Brazilian Symposium on Artificial Intelligence (SBIA 2002) (Lecture Notes in Computer Science, Vol. 2507)*. Springer, Berlin, Heidelberg, 118–128. doi:10.1007/3-540-36127-8_12

[25] Haitong Jiang, Chunlin Liu, Yile Wang, and Yuhong Feng. 2026. Exact Feedback Is Not Control: Evaluating Text-based Closed-Loop Revision in LLMs. arXiv:2609.28150 [cs.CL] doi:10.48550/arXiv.2609.28150

[26] Jingyan Jiang, Huihuo Zheng, Rajeev Thakur, and Chih-Hsuan Yang. 2026. What Confidence Routing Is Actually Doing: Auditing Routing, Calibration, and Commitment in Multi-Agent Deliberation. arXiv:2609.27822 [cs.MA] doi:10.48550/arXiv.2609.27822

[27] Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. 2024. When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs. *Transactions of the Association for Computational Linguistics* 12 (2024), 1417–1440. arXiv:2406.01297 doi:10.1162/tacl_a_00713

[28] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, 611–626. arXiv:2309.06180 doi:10.1145/3600006.3613165

[29] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. 2024. Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Miami, Florida, USA, 17889–17904. arXiv:2305.19118 doi:10.18653/v1/2024.emnlp-main.992

[30] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. 46534–46594. arXiv:2303.17651 doi:10.52202/075280-2019

[31] Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157. doi:10.1007/BF02295996

[32] Jacob M. Montgomery, Brendan Nyhan, and Michelle Torres. 2018. How Conditioning on Posttreatment Variables Can Ruin Your Experiment and What to Do about It. *American Journal of Political Science* 62, 3 (2018), 760–775. doi:10.1111/ajps.12357

[33] Jihwan Oh, Minchan Jeong, Jongwoo Ko, and Se-Young Yun. 2026. From Belief Entrenchment to Robust Reasoning in LLM Agents. *Transactions of the Association for Computational Linguistics* 14 (2026), 1286–1307. arXiv:2503.16814 doi:10.1162/tacl.a.728

[34] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925 [cs.CL] doi:10.48550/arXiv.2508.10925

[35] Abdelghny Orogat, Ana Rostam, and Essam Mansour. 2026. Architectural Design, Not Only Model Intelligence, Governs Multi-Agent LLM Performance. arXiv:2602.03128 [cs.AI] doi:10.48550/arXiv.2602.03128

[36] Dipkumar Patel. 2026. Representational Collapse in Multi-Agent LLM Committees: Measurement and Diversity-Aware Consensus. arXiv:2604.03809 [cs.LG] doi:10.48550/arXiv.2604.03809

[37] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 15174–15186. arXiv:2307.07924 doi:10.18653/v1/2024.acl-long.810

[38] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. 8634–8652. arXiv:2303.11366 doi:10.52202/075280-0377

[39] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. 2025. On the Self-Verification Limitations of Large Language Models on Reasoning and Planning Tasks. In *International Conference on Learning Representations*. arXiv:2402.08115

[40] Michael Stettler, Benjamin Girardet, Jonas Canton, and Nicolas Corod. 2026. Progressive Skill Discovery as Access Control for Tool-Using LLM Agents: Structural Governance through Role-Scoped Capability Delivery. arXiv:2609.28693 [cs.AI] doi:10.48550/arXiv.2609.28693

[41] Milind Tambe. 1997. Towards Flexible Teamwork. *Journal of Artificial Intelligence Research* 7 (1997), 83–124. arXiv:cs/9709101 doi:10.1613/jair.433

[42] Xiaoxuan Wang, Ziniu Hu, Pan Lu, Yanqiao Zhu, Jieyu Zhang, Satyen Subramaniam, Arjun R. Loomba, Shichang Zhang, Yizhou Sun, and Wei Wang. 2024. SciBench: Evaluating College-Level Scientific Problem-Solving Abilities of Large Language Models. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 50622–50649. arXiv:2307.10635

[43] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *Conference on Language Modeling*. arXiv:2308.08155

[44] Chih-Hsuan Yang, Jingyan Jiang, Vikram Vasudevan, Cheng-Hau Yang, Huihuo Zheng, Le Chen, Eliu A. Huerta, Venkatram Vishwanath, Ian T. Foster, and Rajeev Thakur. 2026. Precise but Uncoupled: Reviewer Precision Does Not Guarantee Critique Uptake in Multi-Agent Math Reasoning. arXiv:2607.15388 [cs.AI] doi:10.48550/arXiv.2607.15388

[45] Chih-Hsuan Yang, Jingyan Jiang, Cheng-Hau Yang, Vikram Vasudevan, Huihuo Zheng, Venkatram Vishwanath, and Rajeev Thakur. 2026. LLMs Can Predict Failure Risk, But Struggle to Predict Which Collaboration Protocol Pays Off: Cost-Aware Protocol Routing Across Reasoning Tasks. arXiv:2608.14927 [cs.AI] doi:10.48550/arXiv.2608.14927

[46] Wendi Yu, Lianhao Zhou, Xiangjue Dong, Sai Sudarshan Barath, Declan Staunton, Byung-Jun Yoon, Xiaoning Qian, James Caverlee, and Shuiwang Ji. 2026. When Do Multi-Agent Systems Help? An Information Bottleneck Perspective. arXiv:2607.16133 [cs.LG] doi:10.48550/arXiv.2607.16133

[47] Jiamu Zhang, Lingxi Zhang, Pengjun Lu, Qiyue Zhang, Yu-Neng Chuang, Zhengchen Li, Shuai Xu, Vipin Chaudhary, and Hanjie Chen. 2026. Rethinking the Evaluation of Efficiency Methods for Multi-Agent Systems. To appear at EMNLP 2026. arXiv:2609.05933 [cs.LG] doi:10.48550/arXiv.2609.05933